Deployment & Architecture Documentation

By: Keegan Doyle

1. **How to Deploy the Application**
   - i) Local Development Setup
   - ii) Existing Production Deployment Setup
   - iii) New Production Deployment Setup
2. **Google Cloud Platform (GCP) Handoff**
   - i) Project Overview and Vital Links
   - ii) Ownership & Access Management (IAM)
   - iii) Billing Transfer Protocol
   - iv) Key Infrastructure Dashboard Locations
   - v) CI/CD & Deployment Workflow
   - vi) Routine Maintenance and Monitoring
3. **Necessary Software**
   - i) Core Requirements
   - ii) Administrator Tools and Optional Software
   - iii) Other Technologies
4. **Architecture and External Resources**
   - i) Cloud Infrastructure
   - ii) Integrations and APIs
5. **Security and Configuration Sources**
   - i) Configuration Data
   - ii) Logging Security
   - iii) Infrastructure Level Logging
   - iv) Secret and Credential Sanitization
6. **Dockerfile and docker-compose.yml**
   - i) Dockerfile
   - ii) docker-compose.yml (local development)
7. **Authentication and OAuth Setup Guide**
   - i) Phase 1: Microsoft Entra ID App Registration
   - ii) Phase 2: Generating Credentials and Secrets
   - iii) Phase 3: API Permissions and Admin Consent
   - iv) Phase 4: Django Backend Configuration
   - v) Necessary django-allauth Code and Setup
   - vi) Recommended django-allauth settings

Deployment & Architecture Documentation

By: Keegan Doyle

How to Deploy the Application

This application is containerized with Docker to run identically across different environments. Deployment is divided into Local Development and Production (Google Cloud).

Local Development Setup

1. **Clone the Repository:**

```
git clone
https://github.com/tjbish/InsightersIncWorkflowAutomation
cd InsightersIncWorkflowAutomation
```

   - The above command clones the GitHub repository for local development and opens the project root directory

2. **Configure Environment Variables:** Create a `.env` file in the root directory. Populate the `.env` file with the local database URL, local Django settings module, Django secret key, Debug settings, and developer API keys (Monday.com, Entra ID). Below is a sample `.env` file to include in the project root directory. The `.env` file also can contain a development bypass for testing intake forms. Update the INTAKE_LOGIN_ID and INTAKE_LOGIN_PASSWORD environment variables to change the local intake login bypass.

```
DATABASE_URL=postgres://postgres:postgres@localhost:5432/insighter
s_db
DJANGO_SETTINGS_MODULE=src.config.settings.local
DJANGO_SECRET_KEY=XXX-XXX-XXX
DEBUG=1 # Debug of 1 sets local development to have DEBUG mode
enabled

# Update for the local intake login bypass
INTAKE_LOGIN_ID=admin
INTAKE_LOGIN_PASSWORD=admin

DEV_MONDAY_API=XXX-XXX-XXX
DEV_MONDAY_BOARD_ID=XXXXXXXXXX

# .env
ENTRA_CLIENT_ID=XXX-XXX-XXX
ENTRA_CLIENT_SECRET=XXX-XXX-XXX
ENTRA_TENANT_ID=XXX-XXX-XXX
```

   - After copying this `.env` example file, make sure to update the sample API keys with the actual API keys. The actual API keys will be included in the project handoff.

3. **Add the correct Redirect URI to the Microsoft Entra ID App Registration:** This application uses Microsoft OAuth for secure login and emailing. Please view the documentation on setting up OAuth for setting up a local dev Entra ID tenant and app registration.
4. **Build and Run with Docker:** Run the following command in your terminal

```
docker compose up --build
```

- Docker will automatically download Python, install the `requirements.txt` dependencies, apply database migrations, and start the local server at `http://localhost:8000`

Existing Production Deployment Setup (Existing Google Cloud Platform)

The production deployment is fully automated using a CI/CD pipeline using Google Cloud Build and Google Cloud Run. This documentation is for the existing insighters-app Google Cloud Project. If you need to create a new Google Cloud Project, please view the *New Production Deployment Setup* section.

1. **Infrastructure Prerequisites**: To successfully deploy this application, the target Google Cloud Project must have several core APIs enabled: Cloud Run API, Cloud SQL Admin API, Cloud Build API, Secret Manager API, Artifact Registry API, Compute Engine API. These APIs should already be enabled; however, if they are not currently enabled, you can enable them all at once by running the below command in the Google Cloud CLI. To check if they are enabled, navigate to the APIs & Services app in the Google Cloud Console.

```
gcloud services enable run.googleapis.com sqladmin.googleapis.com
cloudbuild.googleapis.com secretmanager.googleapis.com
artifactregistry.googleapis.com compute.googleapis.com
```

Here is how the platform utilizes each service:

- **Cloud Run API** (run.googleapis.com):
  - **Purpose**: The core hosting environment.
  - **Usage**: This API is required to spin up the serverless containers that run the Django web application. It handles the auto-scaling, HTTP routing, and SSL certificate provisioning for the live site.
- **Cloud SQL Admin API** (sqladmin.googleapis.com)
  - **Purpose**: Database management and secure connectivity.
  - **Usage**: Required to provision the production database instance. This API allows the Cloud Build CI/CD pipeline to use the `exec-wrapper` to securely connect to the database and run automated Django migrations without exposing the database to the public internet.
- **Cloud Build API** (cloudbuild.googleapis.com)
  - **Purpose**: The CI/CD automation engine.
  - **Usage**: This API reads the `cloudbuild.yaml` file whenever new code is pushed to GitHub. It handles the orchestration of building the Docker image, pushing it to the registry, and deploying it to Cloud Run.
- **Secret Manager API** (secretmanager.googleapis.com)
  - **Purpose**: Secure credential storage.
  - **Usage**: Allows the application to dynamically fetch sensitive data at runtime. This prevents hardcoding API keys into the production code.
- **Artifact Registry API** (artifactregistry.googleapis.com)

    - **Purpose**: Docker image storage.
    - **Usage**: Once Cloud Build finishes compiling the application code into a Docker container, it pushes the finished image to the Artifact Registry. Cloud Run then pulls the image from this secure vault to launch the application.
  - **Compute Engine API** (compute.googleapis.com)
    - **Purpose**: Networking and VPC routing.
    - **Usage**: Even though this application is serverless and does not use traditional virtual machines, the Compute Engine API is a mandatory requirement for GCP networking. It is required to establish internal network connections between Cloud Run, Cloud Build, and your Cloud SQL database.
2. **Secrets and Configurations**: Add all production API keys, database credentials, and the Microsoft Entra ID Client Secret to the Google Secret Manager. Update the substitutions block at the bottom of the `cloudbuild.yaml` file with the target environment variables.
3. **Trigger Deployment**: Push your code to the designated main branch on GitHub. Cloud Build will automatically:
    - Build the new Docker container from the Dockerfile in the project repository
    - Push the Docker container image to the Artifact Registry
    - Run Django database migrations securely using the exec-wrapper to connect to Cloud SQL
    - Deploy the image to Cloud Run

New Production Deployment Setup (New Google Cloud Platform)

If you are deploying this application into a brand new Google Cloud Platform (GCP) environment, follow these steps in order. This guide covers creating the project, initializing the core infrastructure, configuring the database, and triggering your first automated build.

Phase 1: Project Creation and Initialization

Before running any scripts, you must setup the foundation of the GCP project.

1. **Create the Project:**
    - Log into the Google Cloud Console.
    - Click the project drop-down at the top and select **New Project**.
    - Name the project (e.g., `insighters-workflow-prod`) and click **Create**.
2. **Enable Billing**:
    - Navigate to **Billing** in the side menu and link the new project to an active billing account. *The required APIs will not activate without a billing account.*
3. **Run the Initialization Script**:
    - Open your local terminal and ensure you have the gcloud CLI installed.
    - Set your active project (the project ID can be found on the GCP home page):

```
gcloud config set project [YOUR_PROJECT_ID]
```

- Run the provided shell script to authenticate, enable all necessary APIs, and provision the Docker Artifact Registry:

```
chmod +x scripts/init_gcp.sh
./scripts/init_gcp.sh
```

Phase 2: Database and Secret Provisioning

The automated pipeline (`cloudbuild.yaml`) expects a live database and a populated Secret Manager to exist before it runs.

1. **Provision Cloud SQL** (PostgreSQL):
   Since the application relies on Cloud SQL for its production database, you must create the instance manually so the CI/CD pipeline can run its automated migrations.
   - Navigate to **Cloud SQL** in the GCP Console.
   - Click **Create Instance** and configure your database. Make sure to use the same database as configured for local testing in the docker-compose file. Currently, the application uses Postgres 15.
   - Once created, create a new database within the instance and note the **Connection Name**
2. **Populate Secret Manager**
   The application strictly reads sensitive credentials from Google Secret Manager to keep them out of the codebase. Navigate to **Secret Manager** and click **Create Secret** for each of the following:
   - DJANGO_SECRET_KEY: (Random Secure String. Utilizing OpenSSL to generate a key is recommended)
   - DATABASE_URL: (Use the URL from the previous step. Example: postgres://[USER]:[PASSWORD]@//cloudsql/[PROJECT_ID]:[REGION]:[INSTANCE_NAME]/[DB_NAME])
   - ENTRA_CLIENT_SECRET: (Contact Insighters IT for the API key)
   - MONDAY_API

Phase 3: Pipeline Permissions (IAM)

For Google Cloud Build to successfully deploy your code, connect to your database, and fetch secrets, its default Service Account needs specific permissions.

1. Navigate to **IAM & Admin > IAM**.
2. Locate the default Cloud Build Service Account (usually formatted as [PROJECT_NUMBER]@cloudbuild.gserviceaccount.com).
3. Edit the account and ensure it has the following Roles assigned:
   - **Cloud Run Admin**: To deploy the new containers.
   - **Service Account User**: Required to act as the runtime service account.

- **Cloud SQL Client**: Required by the `exec-wrapper` to run Django migrations.
- **Secret Manager Secret Accessor**: Required to map secrets into the container.

Phase 4: Configure and Trigger Cloud Build

With the infrastructure initialized, the database running, and secrets stored, you are ready to connect GitHub to GCP and trigger the first deployment.

1. **Connect the Repository**
   - In the GCP Console, navigate to **Cloud Build > Triggers**.
   - Click **Connect Repository** and follow the prompts to authenticate with GitHub and select the InsightersIncWorkflowAutomation repository.
   - Click **Create Trigger**.
   - Set the Event to **Push to a branch**, specify your main deployment branch (e.g., main or production), and set the Configuration type to **Cloud Build configuration file (yaml)**.
2. **Update the Pipeline Variables**
   Before pushing your code, verify the substitutions block at the bottom of your `cloudbuild.yaml` (included in the cloned repo) matches your new GCP environment.

```
substitutions:
    _REGION: 'us-central1'
    _REPO_NAME: 'insighters-repo'  # Must match the repo
created by the init script
    _SERVICE_NAME: 'insighters-workflow-automation'
    _DB_CONNECTION_NAME: 'your-project:us-central1:your-db-
instance'
    _MONDAY_BOARD_ID: 'your-Monday-board-id'
    _ENTRA_CLIENT_ID: 'your-client-id'
    _ENTRA_TENANT_ID: 'your-tenant-id'
```

3. **Deploy**
   Push a commit to your designated GitHub branch. Cloud Build will instantly intercept the push, build the Docker image, run the database schema migrations via Cloud SQL, and deploy the live application URL to Cloud Run.

Deployment & Architecture Documentation

By: Keegan Doyle

Google Cloud Platform (GCP) Handoff

This document outlines the administrative procedures required to manage, monitor, and maintain the Insighters workflow infrastructure on Google Cloud.

Project Overview and Vital Links

- **Project Name**: insighters-app
- **Project ID**: insighters-app
- **Project Number**: 428824878696
- **Live Application URL**: https://insighters-workflow-automation-428824878696.us-central1.run.app
- **Source Code Repository**: https://github.com/tjbish/InsightersIncWorkflowAutomation

Ownership & Access Management (IAM)

- **Primary Project Owner**: Keegan Doyle (keegandoyle26@gmail.com) is the current project owner. The project owner will be John Byrd (jbyrd@technest.cloud). To handoff the GCP to a new owner, navigate to **IAM & Admin > IAM** in the GCP Console**:**
  - Select **Grant Access**
  - Add a new principal (must be a user who has a Google Account or user who is part of a Google Workspace)
  - Assign the `Owner` role to the new principal
  - Either remove the current `Owner` or lower their role to `Editor`
- **Managing Access**: To grant future developers or administrators access, navigate to **IAM & Admin > IAM** in the GCP Console.
  - Recommended Role for Developers: `Editor` (Allows modifying Cloud Run and Cloud Build, but prevents deleting the project or changing billing)
  - Recommended Role for Security: `Secret Manager Admin` (To update application secrets).

Billing Transfer Protocol

(Note: If the project is currently tied to a personal billing account or a temporary one, it must be migrated to the company's account. If Insighters already has an active GCP Billing Account, you can skip directly to the final step to link it to this project.).

1. Navigate to **Billing > Manage billing accounts** in the GCP Console.
2. Click **Create Account**.
3. Name the account (avoid PII) and select your organization if prompted. Choose the country, as this dictates currency and payment options.

4. Create or select a Google Payments Profile, then enter payment details.
5. Navigate to **Billing** > **Account Management** and click **Add project** to link your new billing account to a GCP project.

Key Infrastructure Dashboard Locations

When maintaining the application, the current developers should be familiar with these specific GCP dashboards:

- **Cloud Run**: View active Python/Django container instances, adjust memory allocation, or view real-time traffic metrics.
- **Cloud Build**: Check on the progress of new builds and the deployment of the new builds.
- **Cloud SQL**: Monitor the PostgreSQL database health, view active connections, and manage automated daily backups.
- **Artifact Registry**: The secure vault where the compiled Docker images are stored for deployment.
- **Secret Manager**: The centralized vault for sensitive environment variables.

CI/CD & Deployment Workflow

- **Triggering a Build**: Code pushed to the main branch automatically triggers a deployment via Google Cloud Build.
- **Build Logs**: Navigate to **Cloud Build > History**. If a deployment fails, the logs here will explain the errors and build failures.

Routine Maintenance and Monitoring

The system is serverless and requires minimal routine maintenance, but administrators should be aware of the following:

- **Error Tracking**: Navigate to **Logging > Logs Explorer**. Filter by `Severity: ERROR` to view any application crashes or server issues.
- **Credential Rotation**: If any application secrets or API keys expire, generate a new key, update the value in GCP **Secret Manager**, and manually trigger a new deployment to push the new secrets to the live containers. To view the expiration of secrets, go to the **Secret Manager**, select the secret you are interested in viewing the key expiration, then select **Overview** to view the key expiration.
- **Database Backups**: Cloud SQL is configured to take automated backups daily between 10:00 PM - 2:00 AM. Cloud SQL retains 7 days of database backups. These can be restored directly from the Cloud SQL dashboard in the event of catastrophic data loss.

Deployment & Architecture Documentation

By: Keegan Doyle

Necessary Software

The host machine requires very few localized software because the application is containerized. The application is designed to run identically across all environments.

Core Requirements

- **Docker Desktop / Docker CLI:** Docker is the main software requirement. Docker handles the underlying operating system, the Python 3.11 slim environment, system-level dependencies, and networking. This eliminates the need for complex manual installations on your host machine.
  - **Windows Installation:** Windows | Docker Docs
  - **Linux Installation:** Linux | Docker Docs
  - **Mac Installation:** Mac | Docker Docs
- **Git:** Required to clone the source code repository and trigger the CI/CD pipeline.
  - **Installation Guide (Windows, Linux, and Mac):** Set up Git - GitHub Docs
- **GitHub**: Although it is not a software requirement, a GitHub account is necessary to clone and modify code in the GitHub repository.
  - **Creating an Account**: Creating an account on GitHub - GitHub Docs

Administrator Tools and Optional Software (Recommended):

- **Google Cloud CLI** (gcloud): Useful for administrators who need to view Cloud Logging, manage the Secret Manager, or force manual deployments directly from their local terminal rather than using the GitHub pipeline. Google Cloud CLI is also required to utilize the initialization script. It is highly recommended to download and set up gcloud to take advantage of this automation.
  - **Installation and Setup Guide:** Quickstart: Install the Google Cloud CLI | Google Cloud SDK | Google Cloud Documentation
- **GitHub Desktop**: For those unfamiliar with the Git CLI, GitHub desktop is a powerful and easy to use alternative to manage version control for this application.
  - **Installation Guide**: Installing GitHub Desktop - GitHub Docs

Other Technologies – Downloaded through Docker, No User Action Needed:

Our Docker container downloads the necessary project dependencies through the requirements.txt file in the project repository. The application is built with the Python Django Framework using Django for the backend, PostgreSQL 15 for the database, Docker containerization for the infrastructure, Google Cloud Run for the cloud orchestration, and Django server-side rendering for the frontend.

Deployment & Architecture Documentation

By: Keegan Doyle

Architecture and External Resources

The platform relies on a modern, serverless cloud architecture and integrates with several third-party platforms to handle workflow automation and authentication.

Cloud Infrastructure

- **Google Cloud Platform (GCP)**
  - Services Used: Cloud Run (Serverless Hosting), Cloud SQL (Database), Cloud Build (CI/CD Pipeline), Secret Manager, Artifact Registry (Containerization), and Compute Engine (Networking)
  - For a deeper dive into the GCP services and how to enable the service APIs, please read the Deployment guide.

Integrations and APIs

- **Microsoft Entra ID**
  - **Role:** Handles Single-Tenant OAuth 2.0 Authentication and provides access to the Microsoft Graph API for automated background email sending.
  - **Tier:** Included in standard Microsoft 365 / Azure commercial subscriptions.
- **Monday.com API**
  - **Role:** Automates the creation of items on the intake board and populates custom columns (Emails, Phone Numbers, Locations, Dates).
  - **Tier:** Requires a paid Standard, Pro, or Enterprise Monday.com plan to access the API. Insighters Inc. currently has a Standard plan. All paid plans have access to the API, a future change in the plan will not change the API usage unless indicated by Monday.com.

Deployment & Architecture Documentation

By: Keegan Doyle

Security and Configuration Sources

To maintain enterprise grade security, this application never stores secrets, passwords, or API keys in plain text within the codebase.

Configuration data is pulled from three distinct sources depending on the environment:

1. **Local Development** (`.env` file): When running locally, the application relies on a hidden `.env` file stored on the developer's machine
2. **Production Routing** (Environment Variables): Non sensitive routing information (like URLs, Microsoft Tenant IDs, and Client IDs) is injected directly into the Cloud Run container via the `cloudbuild.yaml` pipeline file.
3. **Production Secrets** (Google Secret Manager): Sensitive passwords, database, credentials, and production API keys are fetched dynamically at runtime from Google Secret Manager

Logging Security

The Insighters Inc. Workflow Automation platform processes Highly Sensitive Personal Information (PII) and financial data. To maintain compliance and protect user privacy, the application employs a multi-layered approach to log sanitization, ensuring that sensitive data is never permanently recorded in server error logs or crash reports.

1. **Global Application-Level Sanitization (Django):**
   - The main defense against data leakage is a global exception filter built directly into the application's backend.
   - If the Django application encounters a fatal error or crashes while processing an intake form, it automatically generates a crash report containing the variables and POST data active at the exact moment of the crash.
   - To prevent this data from being written to Google Cloud Logging, the application utilizes a custom GlobalSensitiveDataFilter
2. **GlobalSensitiveDataFilter:**
   - This filter intercepts the crash report payload before it is rendered or logged. It scans the incoming HTML form input names (POST parameters) and redacts any field that matches a keyword list.
   - If a match is found, the true value is permanently overwritten with: `[REDACTED BY GLOBAL FILTER]`
3. **Protected Keywords:**
   - The filter currently scrubs the following form fields:
     a. `ssn` (Social Security Numbers)
     b. `fin_number` (Federal Identification Numbers)
     c. `routing` (Bank Routing Numbers)
     d. `bank_account` (Bank Account Numbers)
     e. `dl_number` (Driver's License Numbers)

f. `dl_exp` (Driver's License Expiration Dates)
g. `dl_issued` (Driver's License Issue Dates)
h. `password` (Authentication Credentials)

Note: Because this checks if the keyword is contained anywhere within the input name, a field named `business_routing_number` will successfully trigger the redaction.

Infrastructure Level Logging (Google Cloud)

Once the application safely scrubs the payload, the sanitized logs are passed up to the cloud hosting environment.

- **Immutable Storage**: All application print statements, errors, and system warnings are routed to Google Cloud Logging. Once written, the logs are immutable to prevent tampering or unauthorized deletion.
- **Log Retention Policy**: By default, logs are stored in the `_Default` bucket. The retention period for this bucket dictates when Google purges old log entries. (Administrators can adjust this retention window using the Google Cloud CLI). Currently the window is 30 days for log retention.

Secret and Credential Sanitization

- In addition to user PII, the application prevents its own system credentials from leaking into the logs.
- Because the system fetches production credentials (like the Monday.com API Token and Microsoft Entra ID Client Secret) dynamically from Google Secret Manager at runtime, these values are never written to physical environment files or logged during the deployment pipeline. The system holds these secrets in volatile memory, making sure that if a process crashes, the credentials die with the container and are not output to the system logs.
- If the secrets happen to slip through to the logs a DynamicSecretFilter class in the production.py file collects a set of known secrets during the collection of secrets from the secret manager and redacts the message if a known secret appears in the secrets log. This class is contained in the base.py file. This class also uses regex as a fallback to prevent PII such as SSN, email, and OAuth bearer tokens from appearing in the cloud logs.

Deployment & Architecture Documentation

By: Keegan Doyle

Dockerfile and docker-compose.yml

This document provides a deep dive into the structure and setup of both the Dockerfile and docker-compose.yml.

Dockerfile

The Dockerfile acts as the blueprint for our application's environment. This file builds a self-contained, highly secure, and optimized Linux environment.

Here is a line-by-line breakdown of the file:

1. **Base Image and Environment**

```
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
```

- **Foundation**: We use python:3.11-slim. The slim version is crucial because it reduces the amount of unnecessary Debian packages, has a much smaller image size, and a lower attack surface because of the lower number of preinstalled packages.
- **Python Optimizations**: `PYTHONDONTWRITEBYTECODE=1` prevents Python from cluttering the container with .pyc cache files, saving space. `PYTHONUNBUFFERED=1` makes sure that Django's print() statements and server errors stream directly to Google Cloud Logging instead of getting trapped in the memory buffer.

2. **Enterprise Security**

```
RUN addgroup --system appgroup && adduser --system --ingroup appgroup
appuser
```

- **Principle of Least Privilege**: By default, Docker containers run as Root. If a hacker gained access to the Django app, they would have root access to the container. We create a restricted appuser application that has minimal system permissions in the worst case scenario of container compromise.

3. **System and Python Dependencies**

```
RUN apt-get update && apt-get install -y libpq-dev gcc && rm -rf
/var/lib/apt/lists/*

COPY requirements.txt /app/
RUN pip install --no-cache-dir -r requirements.txt
```

- **System Packages:** Because we use PostgreSQL for our database, Linux needs libpq-dev and the gcc compiler to successfully build the psycopg2 Python database adapter.
- **Cleanup:** The `rm -rf /var/lib/apt/lists/*` command deletes the temporary apt-get download cache, shrinking the final image size.

- **No-Cache Pip:** Similarly, `--no-cache-dir` tells pip not to save the downloaded installation files, keeping the image as small as possible.

4. **Code Assembly and Permissions**

```
COPY --chown=appuser:appgroup . /app/

RUN chown appuser:appgroup /app && mkdir -p /app/staticfiles && chown
appuser:appgroup /app/staticfiles

RUN sed -i 's/\r$//' /app/scripts/entrypoint.sh && chmod +x
/app/scripts/entrypoint.sh
```

- **Secure Copy**: The codebase is copied into the container, explicitly transferring ownership to our restricted appuser.
- **Static Files**: We manually create the `/app/staticfiles` directory so Django can collect static assets (like CSS and images) without hitting permission denied errors.
- **The Windows Fix:** The sed command automatically scrubs hidden Windows carriage returns (\r) from the bash script. This prevents the container from crashing if a developer edited the entrypoint script on a Windows machine.

5. **Execution and Cloud Run Optimization**

```
EXPOSE 8080
USER appuser
ENTRYPOINT ["/app/scripts/entrypoint.sh"]
CMD ["sh", "-c", "exec gunicorn --bind :$PORT --workers 1 --threads 8
--timeout 0 src.config.wsgi:application"]
```

- **Port 8080:** This is the standard networking port expected by Google Cloud Run.
- **Switching Users:** Before booting the server, Docker formally switches down from root to the restricted appuser.
- **Entrypoint Script:** The entrypoint.sh script is triggered first, ensuring static files are collected and the server has the right commands passed to it.
- **Gunicorn Tuning:** We do not use the standard python manage.py runserver because it cannot handle multiple concurrent users. Instead, we use Gunicorn, tuned specifically for Cloud Run
  a. `--workers 1 --threads 8`: A single worker with multiple threads is the Google-recommended setup for serverless Python containers.
  b. `--timeout 0`: Cloud Run manages HTTP timeouts at the infrastructure layer, so we disable Gunicorn's internal timeout to prevent the two systems from fighting each other.

docker-compose.yml (local development)

The docker-compose.yml file is the conductor for our local development. It spins up web and database containers and wires them together so developers can test the application without needing to test on the live Google Cloud environment.

1. **The web Service (**The Django Application**)**

Deployment & Architecture Documentation

By: Keegan Doyle

This service runs our Python codebase. We override several production settings to make local development faster and easier.

```
web:
    build: .
    command: sh -c "python manage.py migrate && python manage.py
runserver 0.0.0.0:8000"
```

- **Build Context:** `build: .` tells Compose to build the container using the Dockerfile located in the root directory.
- **Development Override**: In production, our Dockerfile uses Gunicorn. For local development, we override that command. We tell the container to automatically run database migrations and then boot up the standard Django development server (runserver).

```
volumes:
    - .:/app
    - /app/staticfiles
ports:
    - "8000:8000"
```

- **Hot-Reloading (Volumes):** By mounting `.:/app`, we link our laptop's project folder directly to the container's `/app` folder. If a developer edits a Python file or HTML template, the changes reflect inside the running container without needing to rebuild it.
- **Static File Protection:** - `/app/staticfiles` creates an anonymous volume so our local machine does not accidentally overwrite or delete the static files.

```
depends_on:
      db:
        condition: service_healthy
    env_file:
      - .env
    environment:
      -
DATABASE_URL=postgres://postgres:postgres@db:5432/insighters_db
      - DJANGO_SETTINGS_MODULE=src.config.settings.local
```

- **Race Condition Prevention:** The `depends_on` block tells the web service not to start until the database container is completely healthy. This prevents Django from crashing by trying to connect to a database that is still booting up.
- **Environment Configuration:** It loads the local `.env` file but explicitly hardcodes the `DATABASE_URL` to point to our local Docker database, and forces Django to use our local.py settings module instead of production.

2. **The db Service (**The PostgreSQL Database**)**

This service provides a local replica of our production Google Cloud SQL database environment.

```
db:
    image: postgres:15
    ports:
```

```
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
```

- **The Engine:** We use the official postgres:15 to match our Google Cloud SQL Database
- **Persistent Data:** The `postgres_data` volume provides persistence for the container database when the containers shut down. The tables are saved and restored for the next time the containers are built.

```
environment:
    - POSTGRES_DB=insighters_db
    - POSTGRES_USER=postgres
    - POSTGRES_PASSWORD=postgres
healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
    timeout: 5s
    retries: 5
```

- **Automatic Provisioning**: The environment variables automatically create a root database named insighters_db with a default postgres user and password.
- **The Health Check:** Every 5 seconds, Docker uses the `pg_isready` command to ping the database. Docker only marks the container as healthy when the database responds with a "yes". This then signals the web container to safely boot up.

Deployment & Architecture Documentation

By: Keegan Doyle

Authentication and OAuth Setup Guide

The Insighters Inc. Workflow Automation platform uses Microsoft Entra ID for enterprise authentication. The application is configured as a strictly Single Tenant application, meaning only authorized internal employees within the Insighters directory can log in.

The backend relies on the `django-allauth` package to process the OAuth 2.0 flow.

Phase 1: Microsoft Entra ID App Registration (*An existing Microsoft Tenant is a Prerequisite*)

To authorize the application to communicate with Microsoft, an IT Administrator must create an App Registration in the Entra ID Portal. Insighters Inc. has an Entra ID portal through their Microsoft 365 subscription. If you need to setup a dev tenant for testing please follow this guide: https://learn.microsoft.com/en-us/entra/fundamentals/create-new-tenant.

1. **Navigate to the Portal:** Log in to the https://entra.microsoft.com/
2. **Create Registration:**
   - Go to **Entra ID > App registrations > New registration**.
   - **Name**: Insighters Workflow Automation (or similar).
   - **Supported account types**: Select *Single tenant only*. This is the critical setting that blocks external or personal Microsoft accounts from accessing the application.
3. **Setup the Redirect URI:**
   - Under the **Redirect URI** section, select the platform as **Web**
   - Enter the exact callback URL for your environment.
   - Local Development: http://localhost:8000/accounts/microsoft/login/callback/
   - Production: https://[YOUR-CLOUD-RUN-URL]/accounts/microsoft/login/callback/
   - Click **Register**
   - If you need to add an additional Redirect URI or update an existing Redirect URI go to **Manage > Authentication > Add Redirect URI**

Phase 2: Generating Credentials and Secrets

Once the application is registered, you must generate three pieces of information to feed into the Django backend.

1. **Client ID and Tenant ID:**
   - On the App Registration **Overview** page, locate the **Application (client) ID** and the **Directory (tenant) ID**. Save these values.
2. **Generate the Client Secret:**
   - Navigate to **Certificates & secrets > Client secrets.**
   - Click **New client secret.** Add a description (e.g., "Django Prod Secret") and set an expiration date

- **Important:** Immediately copy the Value of the secret. *Microsoft will hide this value permanently once you leave the page.*

Phase 3: API Permissions and Admin Consent

Because the application also uses the Microsoft Graph API to send automated background emails, it requires specific permissions beyond basic user sign-in.

1. Navigate to **API permissions**
2. Click **Add a permission > Microsoft Graph**
3. Add the following permissions:
   - **Delegated Permissions:** User.Read (Allows the app to read the profile of the logging-in user) and offline_access (Maintain access to data you have given it access to)
   - **Application Permissions**: Mail.Send (Allows the server to send emails in the background without a user actively clicking send).
4. Grant Admin Consent:
   - Application permissions require administrator approval. Click the **Grant admin consent for Insighters** button. The status icons next to the permissions should turn into green checkmarks. If this step is skipped, the background email automation will fail.

Phase 4: Django Backend Configuration

The Django backend expects the Microsoft credentials to be passed in as environment variables. The codebase uses `django-allauth` configured to read these dynamically.

Required Environment Variables:

- `ENTRA_CLIENT_ID`: (From Phase 2)
- `ENTRA_CLIENT_SECRET`: (From Phase 2)
- `ENTRA_TENANT_ID`: (From Phase 2)

Where to configure them:

- **Local Development:** Add these three variables directly to your `.env` file.
- **Production Deployment:** Add the `ENTRA_CLIENT_SECRET` to Google Secret Manager. Add the `ENTRA_CLIENT_ID` and `ENTRA_TENANT_ID` to the `cloudbuild.yaml` substitutions block so they are injected into the Cloud Run container at runtime.

Necessary `django-allauth` Code and Setup:

All setup for the `django-allauth` settings should take place in the base.py file.

1. **Setting up the `SOCIAL_ACCOUNT_PROVIDERS` dictionary:**

Deployment & Architecture Documentation

By: Keegan Doyle

```
SOCIALACCOUNT_PROVIDERS = {
    'microsoft': {
        'TENANT': MICROSOFT_TENANT_ID,
        'APP': {
            'client_id': MICROSOFT_CLIENT_ID,
            'secret': MICROSOFT_CLIENT_SECRET,
        },
        'SCOPE': [
            'User.Read',
            'Mail.Send',
            'offline_access',
        ],
        'AUTH_PARAMS': {
            'prompt': 'select_account',
            'access_type': 'offline',
        },
    }
}
```

i. **The Provider Key** (`microsoft`)
   - This tells `django-allauth` which of its built-in social providers you are configuring. Since you are using Microsoft Entra ID, this activates the specific Microsoft Graph API endpoints under the hood

ii. **The Identity Boundary** (`TENANT`)
   - This is the single most important security line in the dictionary. By passing in the Insighters Inc. Tenant ID instead of the default common you are telling Django and Microsoft to only allow Insighters employees to log into the application.

iii. **The Credentials** (`APP`)
   - This is how your application proves its identity to Microsoft. This configures the Client ID and Secret from phase 2.

iv. **The Permissions** (`SCOPE`)
   - This registers the API permission from Phase 3 in the Django backend allowing users to login and send emails.

v. **The Login Behavior** (`AUTH_PARAMS`)
   - These parameters dictate exactly how the Microsoft login screen behaves when the user clicks the "Sign In" button on your site.
   - `prompt: 'select_account'`: If a user is already logged into Microsoft on their browser, this forces Microsoft to display the Pick an account screen. This prevents the browser from automatically trying to log them in with the wrong account and instantly failing.
   - `access_type: 'offline'`: This works in tandem with the `'offline_access'` scope above to guarantee that Microsoft issues the refresh tokens your application needs for background processing.

Deployment & Architecture Documentation

By: Keegan Doyle

Recommended `django-allauth` settings:

1. **Stripping Out Usernames**

```
ACCOUNT_EMAIL_REQUIRED = True
ACCOUNT_USERNAME_REQUIRED = False
ACCOUNT_AUTHENTICATION_METHOD = 'email'
```

- By default, Django's internal database relies heavily on traditional usernames. However, enterprise systems like Microsoft Entra ID use email addresses as the primary identifier.
- This block forces Django to drop the username requirement. It makes sure that when Microsoft passes an employee's data back to your app, Django creates their account using their official @myinsighters.com email address without crashing or asking them to make a username.

2. **Traffic Routing (Logins & Logouts)**

```
LOGIN_REDIRECT_URL = '/dashboard'
LOGIN_URL = '/admin/login/'
ACCOUNT_LOGOUT_REDIRECT_URL = '/'
```

- `LOGIN_REDIRECT_URL`: The exact page the employee is sent to the moment Microsoft successfully validates their identity.
- `LOGIN_URL`: If an unauthenticated person tries to type insighters.com/dashboard into their browser, Django intercepts them and automatically redirects them here to log in.
- `ACCOUNT_LOGOUT_REDIRECT_URL`: When an employee clicks "Log Out," they are sent back to the public homepage (/).

3. **Streamlining the User Experience**

```
ACCOUNT_LOGOUT_ON_GET = True
SOCIALACCOUNT_LOGIN_ON_GET = True
```

- `ACCOUNT_LOGOUT_ON_GET = True`: Bypasses Djang's default "Are you sure you want to log out?" confirmation page. When a user clicks Log Out, they are instantly logged out.
- `SOCIALACCOUNT_LOGIN_ON_GET = True`: This is a UX improvement. By default, django-allauth shows a generic intermediary page that says "You are about to sign in using a third-party account: Microsoft. Click continue." Setting this to True skips that page, sending the user seamlessly from your "Sign In" button straight to the Microsoft login portal.

4. **Background Automation**

```
SOCIALACCOUNT_STORE_TOKENS = True
SOCIALACCOUNT_ADAPTER =
'allauth.socialaccount.adapter.DefaultSocialAccountAdapter'
```

- `SOCIALACCOUNT_STORE_TOKENS = True`: This is a critical setting for the automated emails. When Microsoft approves the login, it sends back an "Access Token" and a "Refresh Token." This setting tells Django to save those tokens in the database. Later, when an intake form is submitted, your backend retrieves these saved tokens to prove to Microsoft Graph API that it has the authority to send an email on behalf of that user.
- `SOCIALACCOUNT_ADAPTER`: This simply tells the system to use the default logic for mapping the Microsoft profile data (like First Name and Last Name) to the internal Django user model.